

Robotics with ROS 2

The Software Backbone of Trinetra-AI

PHASE 10 — RESEARCH-GRADE NOTES

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

Every algorithm built in Phases 4–9 has to *run somewhere*, talk to real sensors, and be simulated, deployed, and debugged. **ROS 2** is that backbone: the middleware that turns a pile of modules into a modular robotic system. This document builds Trinetra-AI as a ROS 2 framework — nodes, topics, DDS, TF2, `robot_localization`, Nav2, Gazebo, RViz2, Docker, and Jetson deployment — always in service of GPS-denied multi-sensor navigation.

Foundations → ROS 2 core → navigation stack → simulation & deployment → full architecture.

Note: this project's diagrams are drawn as native (TikZ) figures rather than Mermaid, so they render directly in the compiled PDF.

Contents

I	Foundations	3
1	Robotics Foundations	3
1.1	What is robotics?	3
1.2	Evolution and the sense-plan-act loop	3
1.3	Application domains & research relevance	3
II	The ROS 2 Framework	4
2	ROS 2 Fundamentals	4
2.1	ROS 1 vs ROS 2: why the rewrite	4
2.2	DDS, discovery, and the communication model	4
2.3	The building blocks	5
2.4	Minimal node (Python & C++) and CLI	5
III	The Navigation Stack	6
3	Robot Navigation	6
3.1	TF2 and the transform tree	6
3.2	Localization: robot_localization, EKF/UKF, odometry sources	7
3.3	Nav2: costmaps, AMCL, behavior trees	7
IV	Simulation, Deployment & AI	8
4	Simulation & Deployment	8
4.1	Gazebo: simulate before hardware	8
4.2	Robot modeling (URDF/Xacro) & visualization (RViz2)	8
4.3	Recording: rosbag2	8
4.4	Deployment: Docker, Jetson, CI/CD	8
5	AI + ROS 2	9
V	System Architecture	10
6	Complete Trinetra-AI ROS 2 Architecture	10
6.1	Message flow & interfaces	10

VI Resources & Practice	11
7 Datasets	11
8 Research Papers	11
9 Practical Implementation	12
9.1 ROS 2 workspace & package layout	12
10 Research Contribution	13
11 Interview & Research Preparation	13
11.1 70 interview questions	13
11.2 50 viva questions	15
11.3 40 research questions	17
11.4 30 coding questions	18
12 Summary, Glossary & Roadmap	18
12.1 Summary	18
12.2 Glossary	19
12.3 Roadmap: ROS 2 → complete Trinetra-AI implementation	19

Part I — Foundations

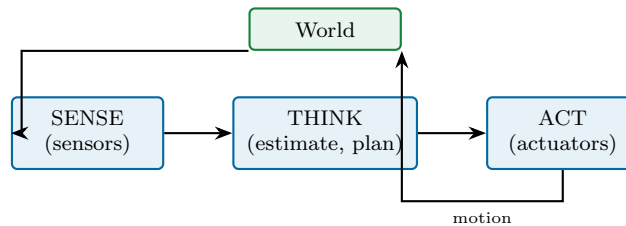
1 Robotics Foundations

1.1 What is robotics?

Robotics is the engineering of machines that *sense*, *think* (compute), and *act* in the physical world. An **autonomous system** closes this *sense–plan–act* loop without human control: sensors perceive state, algorithms estimate and decide, actuators execute. A **mobile robot** adds locomotion, making *where am I?* (localization) and *how do I move there?* (navigation) central — exactly Trinetra-AI’s concern.

Intuition: a robot is a control loop wrapped in hardware. The quality of the “think” stage — state estimation and navigation — determines whether the robot is useful or dangerous. Trinetra-AI is the “think” stage for GPS-denied navigation; ROS 2 is the nervous system that connects it to the “sense” and “act” stages.

1.2 Evolution and the sense–plan–act loop



Robotics evolved from fixed **industrial** arms (repetition in known environments) to **mobile** and **autonomous** systems (reasoning under uncertainty in unknown environments) — the shift that made *state estimation and navigation* the hard core, and made a common software framework (ROS/ROS 2) essential.

1.3 Application domains & research relevance

Domain	Navigation relevance (GPS-denied angle)
Mobile / service robots	indoor SLAM & localization — no GPS indoors
Autonomous vehicles	GNSS+INS+vision; tunnels, urban canyons
Drones / UAVs	lightweight IMU+baro+mag fusion; jamming resilience
Defense systems	passive, unjammable navigation (magnetic/terrain)
Space (ISRO / NASA)	star-tracker+IMU; no GPS beyond MEO
Quantum navigation	quantum-magnetometer + INS + AI (the frontier)

Trinetra-AI is a *robotics software framework* for GPS-denied navigation. Framing it as a ROS 2 system is what makes it modular (swap a sensor or an AI module without touching

the rest), simulatable (test in Gazebo before hardware), deployable (Docker on a Jetson), and *reproducible* — the properties that turn a research prototype into a defensible, extensible thesis contribution.

Part II — The ROS 2 Framework

2 ROS 2 Fundamentals

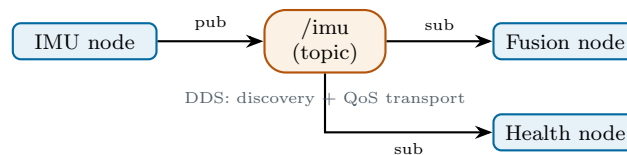
2.1 ROS 1 vs ROS 2: why the rewrite

ROS (Robot Operating System) is not an OS but a *middleware* — a set of libraries and conventions for building robot software as communicating processes. **ROS 2** is a ground-up redesign (Macenski et al., 2022) for *production*: no central master (decentralised discovery), real-time-friendly, multi-platform, secure, and built on **DDS**.

Aspect	ROS 1	ROS 2
Discovery	central roscore master	decentralised (DDS)
Transport	custom TCPROS/UDPROS	DDS (configurable QoS)
Real-time	poor	real-time-friendly
Security	none	SROS2 (auth, encryption)
Multi-robot	fragile	native
Platforms	mainly Linux	Linux/Windows/macOS/RTOS

2.2 DDS, discovery, and the communication model

DDS (Data Distribution Service) is an industry-standard publish–subscribe middleware that ROS 2 sits on. **Nodes** (processes) exchange typed messages over named **topics**; a **publisher** writes, a **subscriber** reads, and DDS handles *discovery* (who is present), transport, and **QoS** (quality of service).



QoS lets each link trade reliability for latency. Key policies: **Reliability** (*reliable* = retransmit until delivered, vs *best-effort* = drop under load); **Durability** (*transient-local* = late subscribers get last message, vs *volatile*); **History** (keep-last-*N* vs keep-all); and **Deadline/ Liveliness**. High-rate sensor streams use *best-effort*, *keep-last* (drop, don't stall); commands and TF use *reliable*.

2.3 The building blocks

Concept	What it is	Trinetra-AI use
Node	a process doing one job	one per sensor / module
Topic	named async pub/sub stream	/imu, /gps, /odom
Service	sync request/response	trigger calibration, get status
Action	long task + feedback + cancel	“navigate to pose”
Parameter	runtime-configurable value	noise covariances, rates
Launch file	start/configure many nodes	bring up the whole stack
Lifecycle node	managed states (configure/activate)	deterministic bring-up
Composition	multiple nodes in one process	zero-copy, low latency on edge
tf2	the transform system	sensor→body→map frames

2.4 Minimal node (Python & C++) and CLI

```
# rclpy publisher: a sensor node streaming IMU on /imu (best-effort, keep-last)
import rclpy; from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from sensor_msgs.msg import Imu
class ImuNode(Node):
    def __init__(self):
        super().__init__("imu_node")
        qos = QoSProfile(reliability=ReliabilityPolicy.BEST_EFFORT,
                        history=HistoryPolicy.KEEP_LAST, depth=10)
        self.pub = self.create_publisher(Imu, "/imu", qos)
        self.create_timer(1/200, self.tick) # 200 Hz
    def tick(self):
        msg = Imu() # fill from hardware...
        self.pub.publish(msg)
def main(): rclpy.init(); rclpy.spin(ImuNode()); rclpy.shutdown()
```

```
// rclcpp equivalent (subscriber): C++ for the real-time fusion node
#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/imu.hpp"
class FusionNode : public rclcpp::Node {
public:
    FusionNode() : Node("fusion_node") {
        sub_ = create_subscription<sensor_msgs::msg::Imu>(
            "/imu", rclcpp::SensorDataQoS(),
            [this](sensor_msgs::msg::Imu::SharedPtr m){ /* predict/update EKF */ });
    }
private: rclcpp::Subscription<sensor_msgs::msg::Imu>::SharedPtr sub_;
};
```

```
ros2 topic list # what streams exist
ros2 topic hz /imu # measured publish rate
ros2 topic echo /imu # inspect messages
ros2 node info /fusion_node # a node's pubs/subs/services
ros2 launch trinetra bringup.launch.py # start the whole stack
```

Trinetra-AI as ROS 2 nodes: each sensor is a node publishing to a topic (/imu, /mag, /gps, /baro); processing, AI, and fusion are subscriber/publisher nodes chained by topics; long operations (“navigate to pose”) are **actions**; calibration is a **service**; noise covariances are **parameters**. On the Jetson, latency-critical nodes are **composed** into one process for

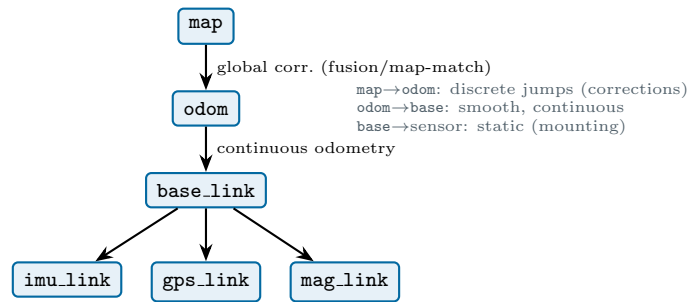
zero-copy. High-rate sensor links use best-effort QoS; TF and commands use reliable. This modularity is exactly what lets a reviewer run one module in isolation.

Part III — The Navigation Stack

3 Robot Navigation

3.1 TF2 and the transform tree

TF2 is ROS 2's system for tracking coordinate frames over time. Every sensor and the robot body is a **frame**; TF2 maintains the time-stamped **transforms** between them as a *tree*, so any measurement can be expressed in any frame. The REP-105 convention chains `map` → `odom` → `base_link` → sensor frames.



A pose is a homogeneous 4×4 transform $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$ (rotation $\mathbf{R} \in SO(3)$, translation $\mathbf{t}$). To express a point known in the sensor frame in the map frame, chain along the tree:

$$\mathbf{T}_{\text{imu}}^{\text{map}} = \mathbf{T}_{\text{odom}}^{\text{map}} \mathbf{T}_{\text{base}}^{\text{odom}} \mathbf{T}_{\text{imu}}^{\text{base}}, \quad \mathbf{p}^{\text{map}} = \mathbf{T}_{\text{imu}}^{\text{map}} \mathbf{p}^{\text{imu}}.$$

The inverse transform is $\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$. The `map`→`odom` link is where Trinetra-AI's *global* corrections (magnetic map-match, GPS) enter — as discrete jumps — while `odom`→`base` stays smooth for control.

```

# Static sensor mounting transform (base_link -> imu_link), published once
from tf2_ros import StaticTransformBroadcaster
from geometry_msgs.msg import TransformStamped
def publish_imu_mount(node):
    b = StaticTransformBroadcaster(node); t = TransformStamped()
    t.header.frame_id = "base_link"; t.child_frame_id = "imu_link"
    t.transform.translation.x = 0.10 # 10 cm forward of body origin
    t.transform.rotation.w = 1.0 # identity orientation
    b.sendTransform(t)

```

3.2 Localization: `robot_localization`, EKF/UKF, odometry sources

`robot_localization` is the standard ROS 2 package for fusing odometry sources into a smooth state estimate (Moore & Stouch, 2014). It provides an **EKF** and a **UKF** node that fuse any number of `Imu`, `Odometry`, and `PoseWithCovariance` inputs — IMU, wheel, visual, and GPS odometry — publishing the fused `odom→base_link` (and, with `navsat_transform`, `map→odom`) transforms.

Odometry source	Provides	Trinetra-AI role
IMU odometry	orientation, angular rate, accel	high-rate backbone
Wheel odometry	velocity from encoders (future)	ground-vehicle aiding
Visual odometry	motion from camera (future)	drift-bounded, texture-rich
GPS (navsat)	absolute position when present	resets drift (map frame)
Magnetic map-match	absolute fix, GPS-denied	the key anchor (custom node)

This maps Phase 7–8 directly onto ROS 2: `robot_localization`’s EKF *is* the fusion core; Trinetra-AI adds a custom magnetic-map-matching node publishing a pose that the EKF fuses like any other absolute source.

3.3 Nav2: costmaps, AMCL, behavior trees

Nav2 is ROS 2’s navigation stack (Macenski et al., “The Marathon 2,” 2020). Its pieces: **AMCL** (Adaptive Monte Carlo Localization — a particle filter localizing in a known map); **costmaps** (grid representations of obstacles/inflation); planners & controllers (global path + local motion); **recovery behaviors** (clearing, spinning when stuck); all orchestrated by a **behavior tree** (a modular, reactive task structure).

How much Nav2 does Trinetra-AI need? Trinetra-AI’s contribution is *localization/state estimation* under GPS denial, not autonomous driving. So the core is `robot_localization` (fusion) + a custom magnetic-map-matching node + TF2 — publishing a robust pose. Nav2 (planning, costmaps, behavior trees) is *optional downstream*: include AMCL/planners only if the demo needs the robot to autonomously move. Keep the thesis focused on the estimate; treat full Nav2 autonomy as integration/future work.

```
# robot_localization EKF config: fuse IMU + wheel odom, 2D nav (ekf.yaml)
ekf_filter_node:
  ros__parameters:
    frequency: 50.0
    two_d_mode: true
    odom_frame: odom
    base_link_frame: base_link
    world_frame: odom
    imu0: /imu
    imu0_config: [false,false,false, true,true,true, false,false,false,
                  true,true,true, true,true,true] # roll,pitch,yaw + rates + accel
    odom0: /wheel/odometry
    odom0_config: [true,true,false, false,false,true, true,false,false,
                  false,false,true, false,false,false]
```


Part IV — Simulation, Deployment & AI

4 Simulation & Deployment

4.1 Gazebo: simulate before hardware

Gazebo is a physics-based 3-D robot simulator (Koenig & Howard, 2004): it models dynamics, sensors (with **noise models**), and environments (**worlds**), so you can develop and test the whole stack without hardware. Note the naming: “Gazebo Classic” (v11) reached end-of-life in Jan 2025; the current line (formerly “Ignition”) is just **Gazebo**, with LTS releases **Fortress** and **Harmonic**.

ROS 2 distro	Paired Gazebo	Notes
Humble (LTS, 2022)	Fortress (official)	most stable; best Jetson support
Iron (2023)	Fortress	
Jazzy (LTS, 2024)	Harmonic (official)	newer LTS; vendor packages
Kilted (2025)	Ionic	latest stable line

For Trinetra-AI, **Humble + Fortress** (stability, Jetson) or **Jazzy + Harmonic** (newer LTS) are the sensible choices. Simulate the IMU/GPS/mag/baro with realistic *noise and bias* so the AI modules train against the same imperfections they will face on hardware.

4.2 Robot modeling (URDF/Xacro) & visualization (RViz2)

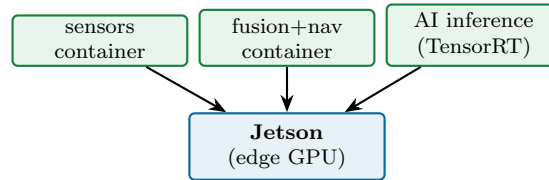
URDF (Unified Robot Description Format) is an XML description of the robot’s **links** (rigid bodies), **joints** (connections), inertial/visual/collision properties, and **sensor** mounts. **Xacro** adds macros/parameters to keep URDF DRY. **RViz2** visualises the robot model, TF frames, paths, point clouds, and trajectories in real time — the primary debugging lens.

4.3 Recording: rosbag2

rosbag2 records and replays topic streams (default format **MCAP**, Phase 11) — the bridge to data engineering: record real or simulated sessions, replay them to test the fusion stack deterministically, and generate training datasets for the AI modules.

4.4 Deployment: Docker, Jetson, CI/CD

Docker packages the ROS 2 stack (code, dependencies, environment) into a portable **image** that runs identically on a laptop or a robot — solving “works on my machine”. **Docker Compose** orchestrates multi-container setups. The **NVIDIA Jetson** is the standard edge GPU compute for on-robot AI inference; **CI/CD** (e.g. GitHub Actions) auto-builds and tests on every commit.



Docker Compose · ROS 2 DDS between containers

```
# Record a simulated GPS-denied session to MCAP for training/replay
ros2 launch trinetra sim_world.launch.py # Gazebo + sensor plugins
ros2 bag record -o denied_run --storage mcap /imu /mag /gps /baro /tf
ros2 bag play denied_run # deterministic replay to test fusion
```

Trinetra-AI's dev-to-deploy loop: model the robot (URDF/Xacro) → simulate with noisy sensors (Gazebo) → record sessions (rosbag2/MCAP) → develop & debug in RViz2 → containerise (Docker) → deploy to Jetson → CI/CD on every commit. This is what lets you validate GPS-denied behaviour *in simulation* (mask the GPS topic to force denial) before risking hardware.

5 AI + ROS 2

The AI modules (Phase 9) deploy as ROS 2 nodes that subscribe to sensor/feature topics, run inference, and publish results. For the edge, models are exported from **PyTorch/TensorFlow** to **ONNX** (a portable model format) and optimised with **TensorRT** (NVIDIA's inference engine: fusion, quantisation to INT8/FP16) for low-latency Jetson inference.

Stage	Tool	Trinetra-AI use
Train	PyTorch / TensorFlow	drift, denoise, confidence models (Phase 9)
Export	ONNX	portable, framework-agnostic model
Optimise	TensorRT (FP16/INT8)	fast Jetson inference
Serve	ROS 2 node (rclpy/rclcpp)	subscribe features → publish result
Cache	Redis (online features)	train/serve parity (Phase 11)

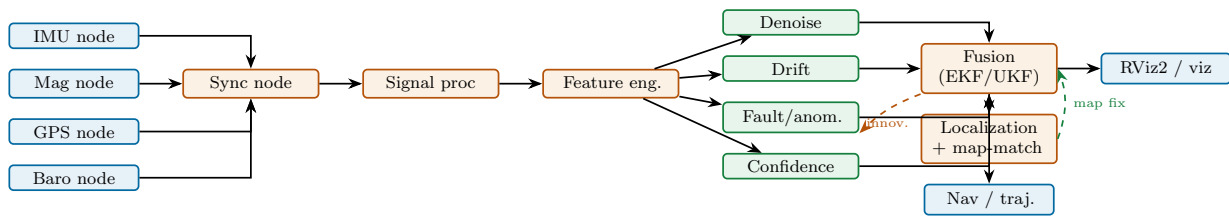
```
# An AI module as a ROS 2 node: subscribe IMU features -> publish drift correction
import rclpy, onnxruntime as ort, numpy as np
from rclpy.node import Node
from std_msgs.msg import Float32MultiArray
class DriftNode(Node):
    def __init__(self):
        super().__init__("drift_node")
        self.sess = ort.InferenceSession("drift_tcn.onnx") # exported model
        self.create_subscription(Float32MultiArray, "/features", self.cb, 10)
        self.pub = self.create_publisher(Float32MultiArray, "/bias_correction", 10)
    def cb(self, msg):
        x = np.array(msg.data, np.float32).reshape(1, 6, -1)
        bias = self.sess.run(None, {"input": x})[0] # inference
        out = Float32MultiArray(data=bias.ravel().tolist()); self.pub.publish(out)
```

Each Phase-9 module becomes a node: `/features` → drift/denoise/anomaly/confidence nodes → their outputs feed the fusion node's adaptive $\mathbf{Q}$, $\mathbf{R}$ and the corrected measurements. Export to ONNX → TensorRT keeps inference within the real-time budget on the Jetson. The clean topic interface means an AI module can be swapped or ablated without touching the fusion core — the modularity that makes the research reproducible.

Part V — System Architecture

6 Complete Trinetra-AI ROS 2 Architecture

Every module from Phases 4–9 becomes a node; topics are the wiring. The graph below is the full system — sensor nodes on the left flow through processing, AI, and fusion to navigation and visualisation, with TF2 threading frames throughout.



6.1 Message flow & interfaces

Node	Subscribes	Publishes
Sensor nodes	(hardware)	<code>/imu</code> , <code>/mag</code> , <code>/gps</code> , <code>/baro</code>
Sync node	all raw sensor topics	<code>/synced</code>
Signal / feature	<code>/synced</code>	<code>/features</code>
AI nodes	<code>/features</code> (+ <code>/innovations</code>)	<code>/clean</code> , <code>/bias</code> , <code>/anomaly</code> , <code>/confidence</code>
Fusion (EKF)	clean + corrections + confidence	<code>/odometry/filtered</code> , TF <code>odom</code> → <code>base</code>
Localization	fused state + <code>/mag_fix</code>	TF <code>map</code> → <code>odom</code>
Nav / trajectory	pose	<code>/plan</code> , <code>/trajectory</code>
Viz	everything	(RViz2 displays)

- **Services:** `/calibrate_mag`, `/reset_filter`, `/get_health`.
- **Actions:** `/navigate_to_pose` (goal + feedback + cancel).
- **Parameters:** noise covariances $\mathbf{Q}$, $\mathbf{R}$, rates, window sizes, frame ids.
- **TF tree:** `map`→`odom`→`base_link`→`{imu,mag,gps,baro}_link`.

This architecture *is* Trinetra-AI as software: Phase 4–6 processing nodes, Phase 9 AI nodes, the Phase 7 fusion node, and the Phase 8 localization/nav nodes, wired by topics with TF2 carrying frames and rosbag2/MCAP (Phase 11) recording everything. The dashed loops are the intelligence (innovations → anomaly; map fix → fusion). Because every block is an independent node with a typed interface, each can be developed, tested, simulated, and ablated on its own — the property that makes the whole thesis modular and reproducible.

Part VI — Resources & Practice

7 Datasets

Real/verified; from a ROS 2 angle, note which ship as (or convert cleanly to) bags. Confirm licence on the official page.

Dataset	Sensors	Ground truth	Diff.	ROS 2 note
EuRoC MAV	IMU + stereo	Vicon/Leica	Med	bag-friendly; drone VIO (Burri 2016)
KITTI	IMU+GPS+LiDAR+camRTK/INS		Med	converts to bag exist (Geiger)
TUM VI	IMU + stereo	partial MoCap	Med	precise sync (Schubert 2018)
RoNIN	IMU (phone)	3D traj.	Med	replay as /imu (Herath 2020)
TLIO	IMU (headset)	VIO	Med	learned inertial (Liu 2020)
OxIOD	IMU (accel/gyro/-mag)	Vicon/VI	Easy–Med	inertial odometry
comma2k19	IMU+GNSS+CAN	GNSS+fusion	Med	driving fusion
MagPIE	IMU + magnetometer	cm-level	Med–Hard	magnetic map-match (Hanley 2017)
DAF-MIT AIA MagNav	aeromagnetic + INS	flight ref.	Hard	magnetic/quantum nav (Gnadt 2023)

ROS 2 workflow: write a small node/script that replays each dataset as ROS 2 topics (/imu, /gps, /mag), so the *exact same* fusion/AI stack runs on recorded data and live hardware. Record your own Gazebo GPS-denied runs as MCAP to complement the public sets — especially for the magnetic-navigation path.

8 Research Papers

Verified, correctly attributed. Priority: ★★★ first, ★★ core, ★ as needed.

Work	Authors, year	Topic	Pri.
Robot Operating System 2: Design, Architecture & Uses in the Wild	Macenski, Foote, Gerkey, Lalancette & Woodall, Science Robotics 2022	the ROS 2 paper	***
ROS: an open-source Robot Operating System	Quigley et al., ICRA-WS 2009	original ROS	**
The Marathon 2: A Navigation System (Nav2)	Macenski, Martín, White & Clavero, IROS 2020	Nav2	***
Survey of modern mobile robotics algorithms in ROS 2	Macenski, Moore, Lu, Merzlyakov & Ferguson, RAS 2023	Nav2 algorithms	**
Impact of ROS 2 Node Composition in Robotic Systems	Macenski, Soragna, Carroll & Ge, RA-L 2023	composition/perf.	*
A Generalized EKF Implementation for ROS (robot_localization)	Moore & Stouch, IAS-13 2014	fusion package	***
Monte Carlo Localization for Mobile Robots (AMCL)	Dellaert, Fox, Burgard & Thrun, ICRA 1999	particle-filter loc.	**
Design and Use Paradigms for Gazebo	Koenig & Howard, IROS 2004	simulator	**
SLAM Toolbox: SLAM for the Dynamic World	Macenski & Jambrecic, JOSS 2021	2-D SLAM	*
Probabilistic Robotics	Thrun, Burgard & Fox, 2005	estimation/SLAM text	***

Reading path: Macenski et al. 2022 (ROS 2 design) → Moore & Stouch (robot_localization, the fusion package you will actually use) → Macenski Nav2 → Thrun for the estimation theory underneath. **Citation hygiene:** the ROS 2 / Nav2 citations here were verified against official sources; still confirm DOIs before a report.

9 Practical Implementation

9.1 ROS 2 workspace & package layout

```
trinetra_ws/ # colcon workspace
|- src/
| |- trinetra_bringup/ # launch files, top-level params
| |- trinetra_msgs/ # custom msgs/srv/action (Health, Confidence)
| |- trinetra_sensors/ # sensor driver nodes -> /imu /mag /gps /baro
| |- trinetra_pipeline/ # sync, signal proc, feature nodes
| |- trinetra_ai/ # Phase 9 modules as nodes (ONNX/TensorRT)
| |- trinetra_fusion/ # robot_localization config + mag map-match node
| |- trinetra_nav/ # optional Nav2 config
| |- trinetra_description/ # URDF/Xacro, meshes
| +- trinetra_sim/ # Gazebo worlds, sensor plugins
|- docker/ # Dockerfile, compose.yaml (Jetson + dev)
|- config/ # ekf.yaml, qos.yaml, sensor calibrations
+- .github/workflows/ # CI: colcon build + test on push
```

```
# bringup.launch.py: start the whole Trinetra-AI stack
from launch import LaunchDescription
from launch_ros.actions import Node
def generate_launch_description():
    return LaunchDescription([
        Node(package="trinetra_sensors", executable="imu_node"),
```

```

Node(package="trinetra_pipeline", executable="sync_node"),
Node(package="trinetra_ai", executable="drift_node"),
Node(package="robot_localization", executable="ekf_node",
      parameters=["config/ekf.yaml"]),
Node(package="trinetra_fusion", executable="mag_mapmatch_node"),
])

```

Build / test / deploy. `colcon build --symlink-install` → `colcon test` (unit + launch tests) → containerise (`docker build`) → `docker compose up` on the Jetson → CI on every push. **Debugging checklist:** `ros2 topic hz` (rates OK?), `ros2 topic echo` (data sane?), `ros2 run tf2_tools view_frames` (TF tree connected?), RViz2 (frames/paths correct?), `ros2 bag record`→`replay` (reproduce a bug deterministically), check QoS mismatches (a reliable subscriber won't hear a best-effort publisher's history the way you expect). **Common mistakes:** QoS incompatibility (silent no-messages); broken/rootless TF tree; wrong `frame_id`; unsynced timestamps; simulating sensors without noise (models overfit to clean data).

10 Research Contribution

ROS 2-flavoured, M.Tech-thesis-scale contributions: (1) *An open, modular GPS-denied navigation stack* — a reusable ROS 2 package set (`robot_localization` + a magnetic-map-matching node + AI nodes) that others can build on; the packaging itself is a contribution. (2) *Custom magnetic-map-matching localization node* publishing a `map`→`odom` correction — the GPS-denied equivalent of `navsat_transform`. (3) *Edge-deployed AI fusion* — ONNX/TensorRT AI nodes hitting real-time budgets on Jetson, benchmarked. (4) *Digital twin* — a Gazebo GPS-denied world for reproducible, ground-truthed evaluation and dataset generation.

Research gaps & future directions. *Cloud/edge robotics* — offload heavy map-matching/training to the cloud, keep the real-time loop on the edge. *Digital twins* — a high-fidelity simulated Trinetra-AI for continuous testing. *Quantum-navigation integration* — a simulated quantum-magnetometer sensor plugin feeding the same fusion node, measuring drift reduction. *Standardised GPS-denied benchmark* — a Gazebo world + MCAP dataset + metrics that others can reuse. The defensible thesis packages Phases 4–9 as a clean, simulatable, edge-deployable ROS 2 framework with the magnetic-inertial GPS-denied contribution at its core — rigorously evaluated in simulation and (if possible) on hardware.

11 Interview & Research Preparation

11.1 70 interview questions

1. What is robotics?
2. Sense-plan-act loop.
3. What is an autonomous system?
4. Mobile vs industrial robot.
5. Why is state estimation the hard core?
6. What is ROS?
7. ROS 1 vs ROS 2.

8. Why did ROS 2 replace ROS 1?
9. What is DDS?
10. What is middleware?
11. Decentralised discovery.
12. What is a node?
13. What is a topic?
14. Publisher vs subscriber.
15. Service vs topic.
16. Action vs service.
17. What is a parameter?
18. What is a launch file?
19. Lifecycle node states.
20. Node composition.
21. What is QoS?
22. Reliability policy.
23. Durability policy.
24. History (keep-last/all).
25. QoS for high-rate sensors.
26. QoS for TF/commands.
27. What is a workspace?
28. colcon build.
29. What is a package?
30. Custom messages.
31. What is TF2?
32. The transform tree.
33. map vs odom vs base_link.
34. Static vs dynamic transform.
35. REP-105 convention.
36. Why map→odom jumps.
37. Transform composition.
38. Inverse transform.
39. What is robot_localization?
40. EKF vs UKF node.
41. Odometry sources.
42. navsat_transform.
43. What is Nav2?
44. What is AMCL?
45. What is a costmap.
46. Recovery behaviors.

47. Behavior trees.
48. Does Trinetra-AI need full Nav2?
49. What is Gazebo?
50. Gazebo Classic vs new Gazebo.
51. Fortress vs Harmonic pairing.
52. Sensor noise models.
53. What is URDF?
54. Xacro.
55. Links vs joints.
56. What is RViz2?
57. What is rosbag2?
58. MCAP vs SQLite3.
59. Why Docker?
60. Docker Compose.
61. What is a Jetson?
62. ONNX.
63. TensorRT.
64. Edge AI.
65. AI module as a node.
66. CI/CD for ROS 2.
67. SROS2 security.
68. Intra-process comms.
69. Debugging with ros2 CLI.
70. Simulate GPS denial.

11.2 50 viva questions

1. Explain the DDS pub/sub model.
2. Derive transform composition along the TF tree.
3. Write the inverse homogeneous transform.
4. Why is map→odom discrete but odom→base smooth?
5. Configure a robot_localization EKF.
6. Map Phase 7 fusion onto ROS 2.
7. Design the Trinetra-AI node graph.
8. Which QoS for each link & why.
9. Diagnose a QoS-mismatch no-message bug.
10. Build the TF tree for the sensors.
11. Publish a static sensor mount.
12. Add a custom magnetic-map-match node.
13. Fuse it via robot_localization.

14. When to use an action vs service.
15. Design the calibration service.
16. Lifecycle bring-up order.
17. Compose nodes for edge latency.
18. Simulate sensors with realistic noise.
19. Choose ROS 2 distro + Gazebo pairing.
20. Record & replay a session.
21. Generate a training dataset in sim.
22. Export a model to ONNX.
23. Optimise with TensorRT.
24. Serve a model as a node.
25. Real-time budget on Jetson.
26. Containerise the stack.
27. Compose multi-container deploy.
28. CI pipeline for colcon.
29. Debug a broken TF tree.
30. Interpret ros2 topic hz.
31. Handle multi-rate sync in ROS 2.
32. Message-flow for the whole stack.
33. Innovations feedback loop as topics.
34. Map-fix feedback to fusion.
35. Parameters vs hardcoding.
36. URDF for the robot.
37. Xacro macros.
38. RViz2 debugging workflow.
39. Nav2 behavior-tree structure.
40. AMCL vs the magnetic map-match PF.
41. Why simulate before hardware.
42. SROS2 in a defense context.
43. Digital-twin design.
44. Cloud/edge split.
45. Benchmark the fusion node.
46. Ablate an AI node.
47. Reproducibility via bags.
48. Failure modes of the stack.
49. Scope: what to include vs defer.
50. Explain the full architecture.

11.3 40 research questions

1. Packaging Phases 4–9 as a reusable ROS 2 stack.
2. A magnetic-map-match localization node as a contribution.
3. Edge real-time limits for AI fusion.
4. TensorRT quantisation vs accuracy for nav models.
5. Digital twin for GPS-denied evaluation.
6. Standard GPS-denied ROS 2 benchmark.
7. Quantum-magnetometer sensor plugin (sim).
8. Cloud/edge map-matching split.
9. Composition vs separate-process latency.
10. QoS tuning for high-rate fusion.
11. Deterministic replay for reproducible research.
12. Multi-robot GPS-denied nav.
13. SROS2 for secure defense nav.
14. Fault-tolerant node architecture.
15. Sim-to-real gap for learned nav nodes.
16. Lifecycle-managed graceful degradation.
17. Latency budgeting across the graph.
18. TF accuracy vs fusion accuracy.
19. Online calibration as a service.
20. Hardware-in-the-loop testing.
21. Jetson power/perf trade-offs.
22. Benchmark robot_localization vs custom fusion.
23. Behavior-tree nav under denial.
24. Magnetic-SLAM node.
25. Dataset generation via Gazebo domain randomisation.
26. CI-driven regression testing of nav accuracy.
27. Message-drop robustness.
28. Time-sync across distributed nodes.
29. Edge vs cloud AI placement.
30. Reproducible experiment tracking in ROS 2.
31. Nav2 plugin for magnetic navigation.
32. Observability with intermittent aiding in ROS 2.
33. Real-time-safe AI inference.
34. Containerised reproducibility.
35. Digital-twin fidelity vs cost.
36. Open-sourcing the stack (impact).
37. Standardised metrics for GPS-denied nav.
38. Multi-sensor plug-and-play interface.
39. Quantum-ready architecture.
40. Thesis scope vs production scope.

11.4 30 coding questions

1. Write an rclpy IMU publisher (200 Hz).
2. Write an rclcpp subscriber.
3. Set best-effort keep-last QoS.
4. Define a custom `Health.msg`.
5. Publish a static TF (base→imu).
6. Look up a transform with tf2.
7. Compose two transforms.
8. Write a bringup launch file.
9. Configure robot_localization EKF (YAML).
10. Node that fuses IMU+wheel odom.
11. Custom magnetic-map-match node publishing a pose.
12. Sync node aligning multi-rate topics.
13. Feature node → `/features`.
14. AI node running an ONNX model.
15. Export a PyTorch model to ONNX.
16. Service to trigger calibration.
17. Action server “navigate to pose”.
18. Record topics to MCAP.
19. Replay a bag & test fusion.
20. Gazebo sensor plugin with noise.
21. URDF with an IMU link.
22. Xacro-parameterise the URDF.
23. RViz2 config for TF+path.
24. Dockerfile for the stack.
25. docker compose for Jetson.
26. GitHub Actions colcon build.
27. ros2 CLI: inspect a topic’s rate.
28. view_frames to dump the TF tree.
29. Mask the GPS topic to force denial.
30. Benchmark fusion-node latency.

12 Summary, Glossary & Roadmap

12.1 Summary

ROS 2 is the software backbone that turns Trinetra-AI’s algorithms into a real, modular robotic system. Built on **DDS** pub/sub with **QoS**, it organises the project as **nodes** exchanging typed messages over **topics**, with **TF2** threading coordinate frames (`map`→`odom`→`base_link`→`sensors`) and **robot_localization** providing the EKF/UKF fusion core. **Gazebo** simulates the robot and noisy sensors before hardware; **RViz2** visualises; **rosbag2/MCAP** records; **Docker** and

the **Jetson** deploy it to the edge; and the Phase-9 AI modules run as **ONNX/TensorRT** inference nodes. The guiding principle is *modularity*: every capability is an independent, testable, swappable node — which is exactly what makes the whole framework simulatable, deployable, and reproducible. Scope stays disciplined: **robot_localization** + a custom magnetic-map-match node is the core; full Nav2 autonomy is optional downstream.

12.2 Glossary

Action — long task with feedback/cancel. **AMCL** — adaptive Monte Carlo (particle) localization. **DDS** — the pub/sub middleware under ROS 2. **Gazebo** — physics robot simulator (Fortress/Harmonic). **Lifecycle node** — node with managed states. **Nav2** — ROS 2 navigation stack. **Node / topic** — process / named pub-sub stream. **QoS** — per-link reliability/durability/history settings. **robot_localization** — EKF/UKF sensor-fusion package. **rosbag2 / MCAP** — recording system / default log format. **TensorRT / ONNX** — edge inference optimiser / portable model format. **TF2** — coordinate-frame transform system. **URDF / Xacro** — robot description format / its macro layer.

12.3 Roadmap: ROS 2 → complete Trinetra-AI implementation

Ties together	Via ROS 2
Phases 4–6 (process)	sync / signal / feature nodes on <code>/synced</code> , <code>/features</code>
Phase 7 (fusion)	robot_localization EKF/UKF node
Phase 8 (navigation)	localization + custom magnetic-map-match node (<code>map</code> → <code>odom</code>)
Phase 9 (AI)	ONNX/TensorRT inference nodes feeding adaptive fusion
Phase 11 (data)	rosbag2/MCAP recording; dataset generation
Deployment	Docker on Jetson; Gazebo digital twin; CI/CD

The whole system, running. This phase is where Trinetra-AI stops being a set of notebooks and becomes a *robot*: sensors publishing, AI nodes inferring, the fusion core estimating, TF2 carrying frames, Gazebo simulating denial, and Docker shipping it to a Jetson. Everything from Phases 4–11 plugs into this ROS 2 skeleton. What remains is implementation and evaluation — building the nodes, generating the GPS-denied benchmark in simulation, and measuring the magnetic-inertial navigation contribution end to end.

End of Phase 10 notes. Trinetra-AI now has its software backbone: sensors → AI → fusion → navigation, as a modular, simulatable, edge-deployable ROS 2 system.